

High-Performance
Computing Center
Stuttgart

OpenFOAM HPC Challenge 2 – HLRS contribution

Flavio C. C. Galeazzo

Supercomputing in Stuttgart

- HLRS was established in 1996 as Germany's first national HPC center
- Facility of the University of Stuttgart
- Scientific as well as industrial use of computing infrastructures
- Focus on engineering and applied sciences
- Coordination and participation in many national and international research projects



User Profile 2025

- The system usage in 2025 shows the clear trend in HLRS for engineering applications
- CFD (Computational Fluid Dynamics) being the most prominent
- But no OpenFOAM ☹ - efficient GPU version needed

System Usage by Scientific Discipline (Hunter)	
CFD	57,24 %
Physics	40,71 %
Computer Science	1,61 %
Transportation and Climate	0,23 %
Reactive Flows	0,15 %
Materials Simulation & Materials Research	0,04 %
Chemistry	0,02 %
Solid State Physics	0,01 %
Other	< 0,01 %
Structural Mechanics	< 0,01 %

https://www.hlrs.de/about/profile/annual-report/

Current flagship system

- Hunter - HPE Cray EX4000
- Theoretical peak performance 48.1 PFlops
- Node types
  - APU AMD Instinct MI300A, 188 nodes
  - CPU AMD EPYC 9374F, 256 nodes
- Memory technology
  - APU 512 GB HBM3 (~5.3 TB/s)
  - CPU 768 GB DDR5-4800



Hardware track

Hardware track

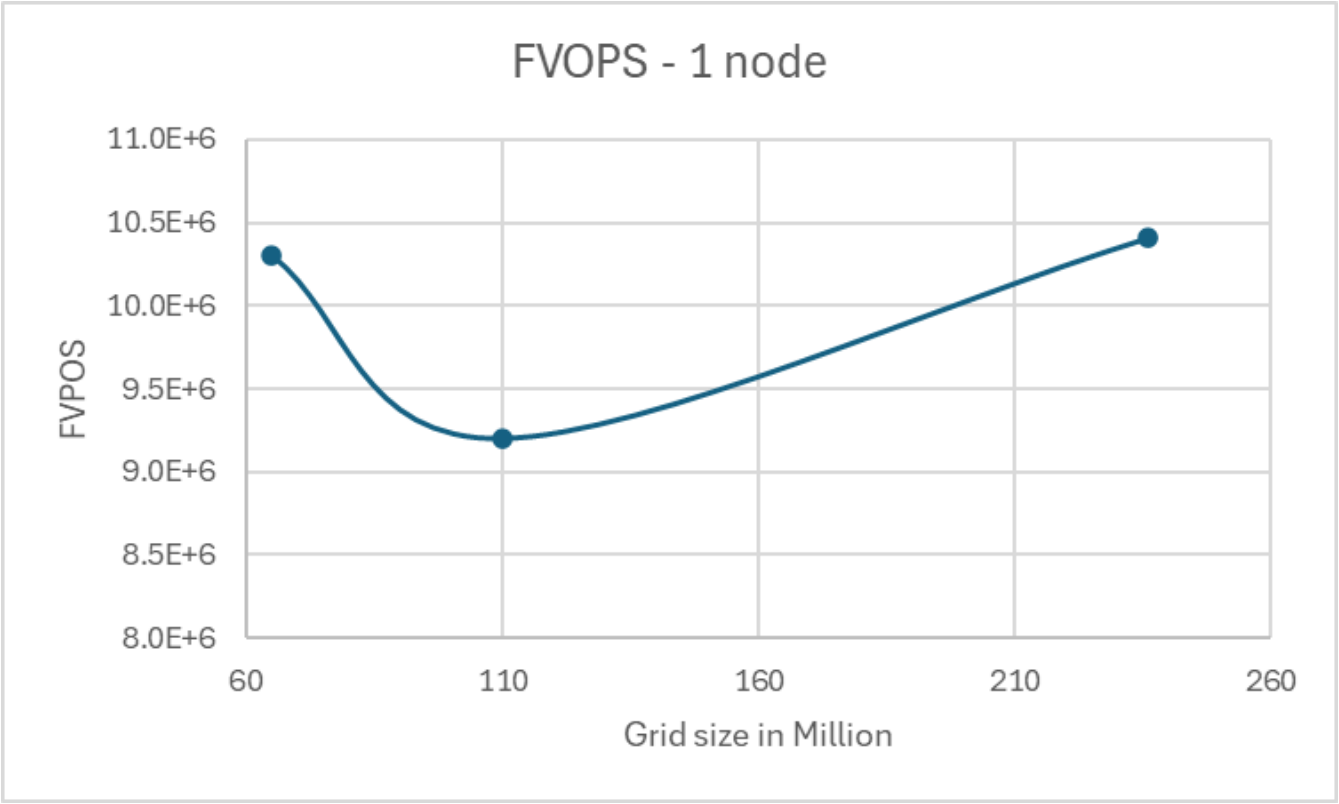
- Compute nodes with 2x CPU AMD EPYC 9374F
  - 768 GB DDR5-4800
- Used as baseline for comparison with software track (GPU OpenMP offloading)

Hardware track

Simulation	Node type	Nodes	APUs/CPUs	Total sim time for 4000 iterations [s]	Time per iteration [s]	Job energy - node [Wh]	Job energy - system [Wh]	Job energy - infra [Wh]	Job energy - node + sys [kWh]	FVOPS
65M	Genoa CPU	1	64	25250	6.30741981	3868	2000	311	5.868	1.03E+07
65M	Genoa CPU	2	128	13018	3.25116169	4138	2139	333	6.277	2.00E+07
65M	Genoa CPU	4	256	6240	1.55755008	4108	2124	330	6.232	4.17E+07
65M	Genoa CPU	8	512	2844	0.70930599	3930	2032	316	5.962	9.16E+07
110M	Genoa CPU	1	64	47876	11.9601882	7234	3740	582	10.974	9.20E+06
236M	Genoa CPU	1	64	90777.25	22.6751	13072	6758	1051	19.83	1.04E+07

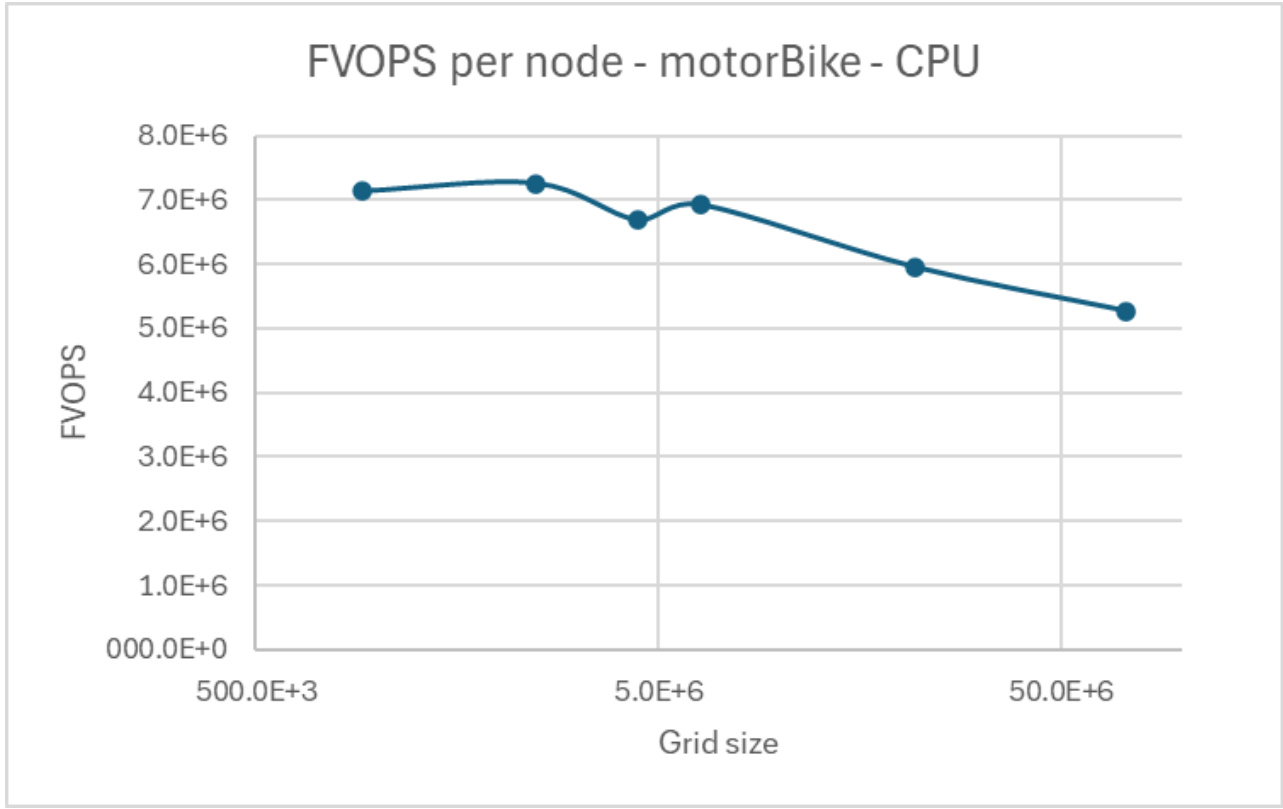
Hardware track

- As all three grids are large for a single CPU node, the throughput (FVOPS) is pretty close



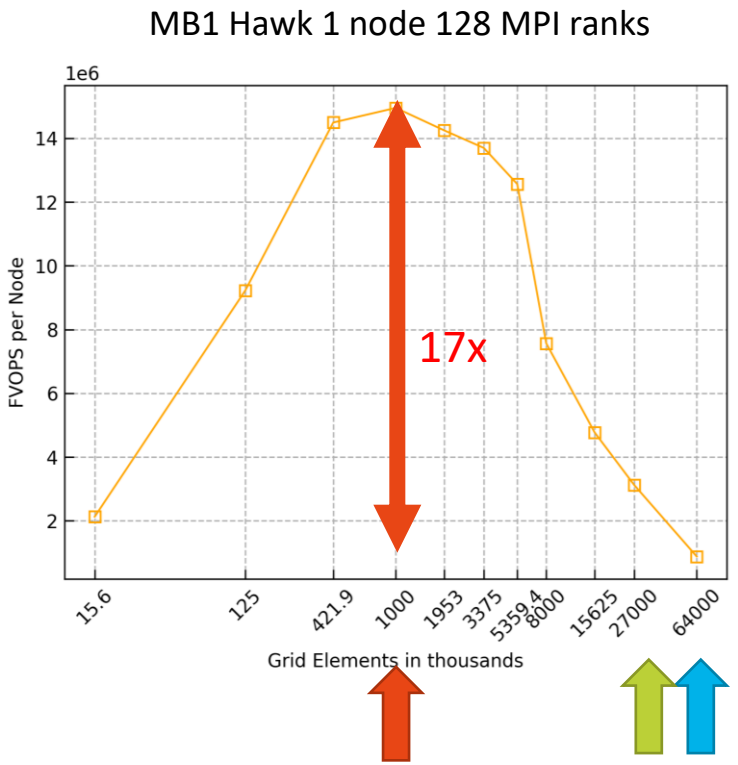
Hardware track

- I tried to produce DrivAer grids with different sizes, but snappyHexMesh was not my friend ☹
- I used then the motorBike test case, as an approximation.



Hardware track

- As a comparison, using the lid-drive cavity test case and the Hawk supercomputer, the differences were much more pronounced



Grid elements in thousand	FVOPS	Speedup in 1 node
64000	881,543	1.00
27000	3,121,026	3.54
1000	14,950,166	16.96

Software track

Software track

- Nodes with 4x APU AMD Instinct MI300A, 512 GB HBM3 (~5.3 TB/s)
- Software version: OpenFOAM GPU porting using OpenMP offload (AMD/Keysight), branch v2412-amd
- I am happy to announce that this code is now publicly available (thanks Mattijs!):

  https://gitlab.com/openfoam/gpu/openfoam-openmp

- The developers have a brand new paper about this code

- K. Saurabh, L. Grinberg, M. Janssens and A. Heather, "Optimizing OpenFOAM for Systems with CPUs and GPUs and Unified Memory Architecture," ISC High Performance 2026 Research Paper Proceedings (41st International Conference), Hamburg, Germany, 2026, pp. 1-13, doi: 10.23919/ISC.2026.11520483.

Software track

Simulation	Setup	Node type	Nodes	CPX?	APUs/CPUs	Total sim time for 4000 iterations [s]	Time per iteration [s]	Job energy - node [Wh]	Job energy - system [Wh]	Job energy - infra [Wh]	Job energy - node + sys [kWh]	FVOPS
65M	old	Mi300A APU	1	no	4	8389	2.05985088	4314	1885	329	6.199	3.16E+07
65M	old	Mi300A APU	1	yes	24	5621	1.39291354	3805	1663	290	5.468	4.67E+07
110M	old	Mi300A APU	1	yes	24	9661	2.39424926	6670	2915	508	9.585	4.59E+07
236M	old	Mi300A APU	2	yes	48	10280	2.5409488	14104	6164	1074	20.268	9.29E+07
65M	paper	Mi300A APU	1	yes	24	2823	0.69154981	1964	858	150	2.822	9.40E+07
110M	paper	Mi300A APU	1	yes	24	4522	1.10602928	3157	1379	240	4.536	9.95E+07
236M	paper	Mi300A APU	1	yes	24	13992	3.44781296	8500	3715	647	12.215	6.84E+07
236M	paper	Mi300A APU	2	yes	48	4987	1.21797826	6810	2976	519	9.786	1.94E+08
65M	paper2	Mi300A APU	1	yes	24	3131	0.76974457	2166	946	165	3.112	8.44E+07
110M	paper2	Mi300A APU	1	yes	24	5139	1.25447416	3418	1494	260	4.912	8.77E+07
236M	paper2	Mi300A APU	1	yes	24	11138	2.73405319	7071	3090	539	10.161	8.63E+07
236M	paper2	Mi300A APU	2	yes	48	5541	1.34923585	7355	3214	560	10.569	1.75E+08
65M	opt	Mi300A APU	1	yes	24	3000	0.73998489	2101	918	160	3.019	8.78E+07
110M	opt	Mi300A APU	1	yes	24	4497	1.10560203	3202	1399	244	4.601	9.95E+07
236M	opt	Mi300A APU	1	yes	24	8846	2.17370493	6174	2698	470	8.872	1.09E+08
236M	opt	Mi300A APU	2	yes	48	5368	1.31791812	7203	3148	549	10.351	1.79E+08

Software track – Optimisations

- The optimisations were proposed by Kumar Saurabh and Leopold Grinberg from AMD: Thank you!
- Use of CPX mode
- In CPX mode, each of the six Accelerator Complex Dies (XCDs) acts as a separate, independent logical GPU instance rather than grouping them together.
- 24 MPI ranks per node – CPX
- 4 MPI ranks per node – SPX

- 1.5x faster with the 65M grid – from 2.06s to 1.39s per iteration

Software track – Optimisations

- Number of patches
- The number of patches has a big impact in the performance when using a GPU
- Concatenated all wall patches into one: reduce the total number of patches to 6 (from 55)
- 2x faster with the 65M and 110M grids – from 2.06s to 1.39s per iteration
- But not much faster with the 236M grid

Software track – Optimisations

- Selection of linear solvers
- Different linear solvers are a better choice for CPU or GPU

For 110M and 236M cases

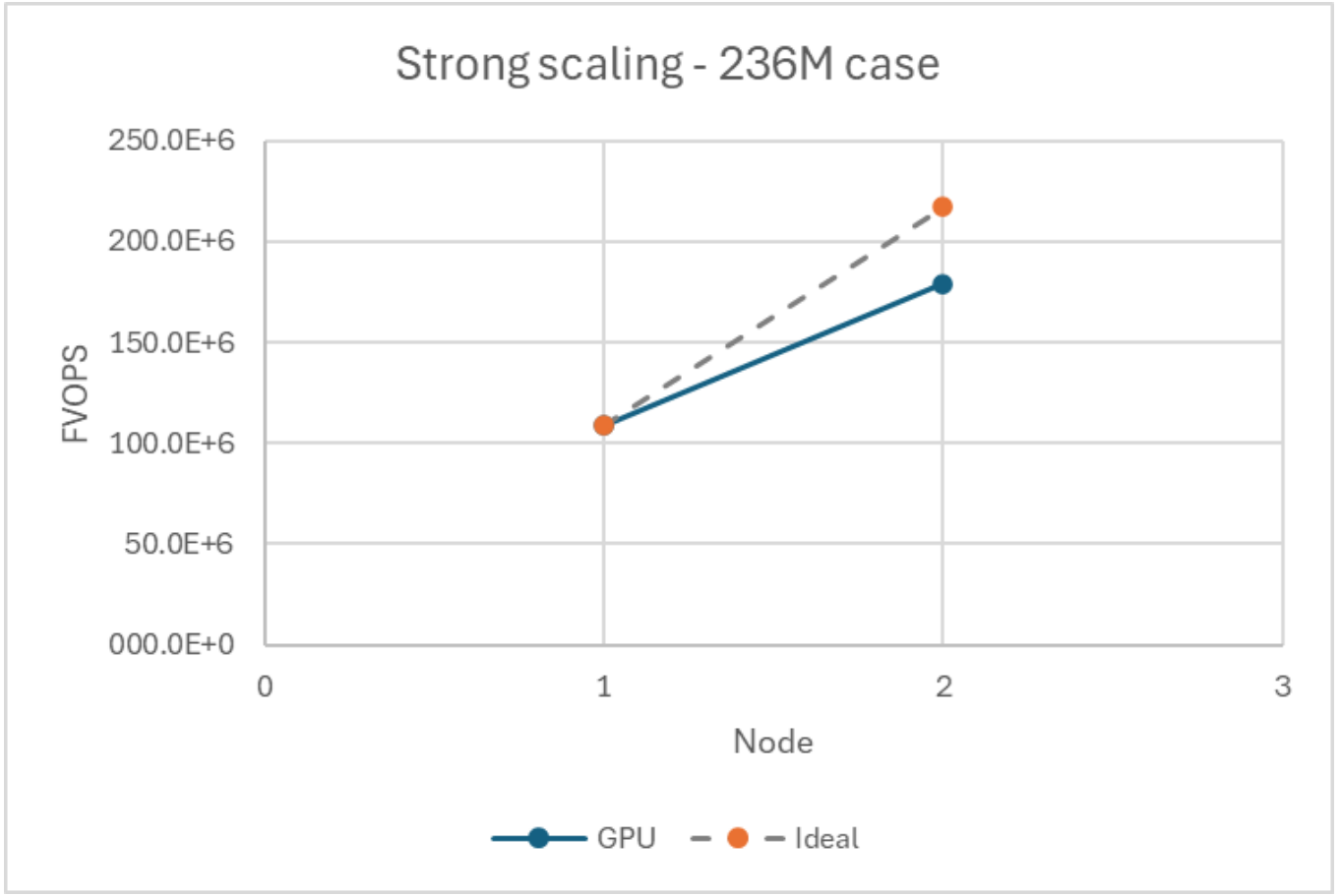
```
U,k,omega
solver          PBiCGStab;
preconditioner  diagonal;
tolerance       1e-8;
relTol          0.1;
```

```
p
solver    GAMG;
tolerance 1e-7;
relTol    0.01;
smoother  twoStageGaussSeidel;
```

```
nPreSweeps              0;
nPostSweeps             2;
postSweepsLevelMultiplier 1;
maxPostSweeps           4;
nFinestSweeps           2;
interpolateCorrection   true;
cacheAgglomeration      true;
nCellsInCoarsestLevel   100;
agglomerator            algebraicPair;
mergeLevels             2;
```

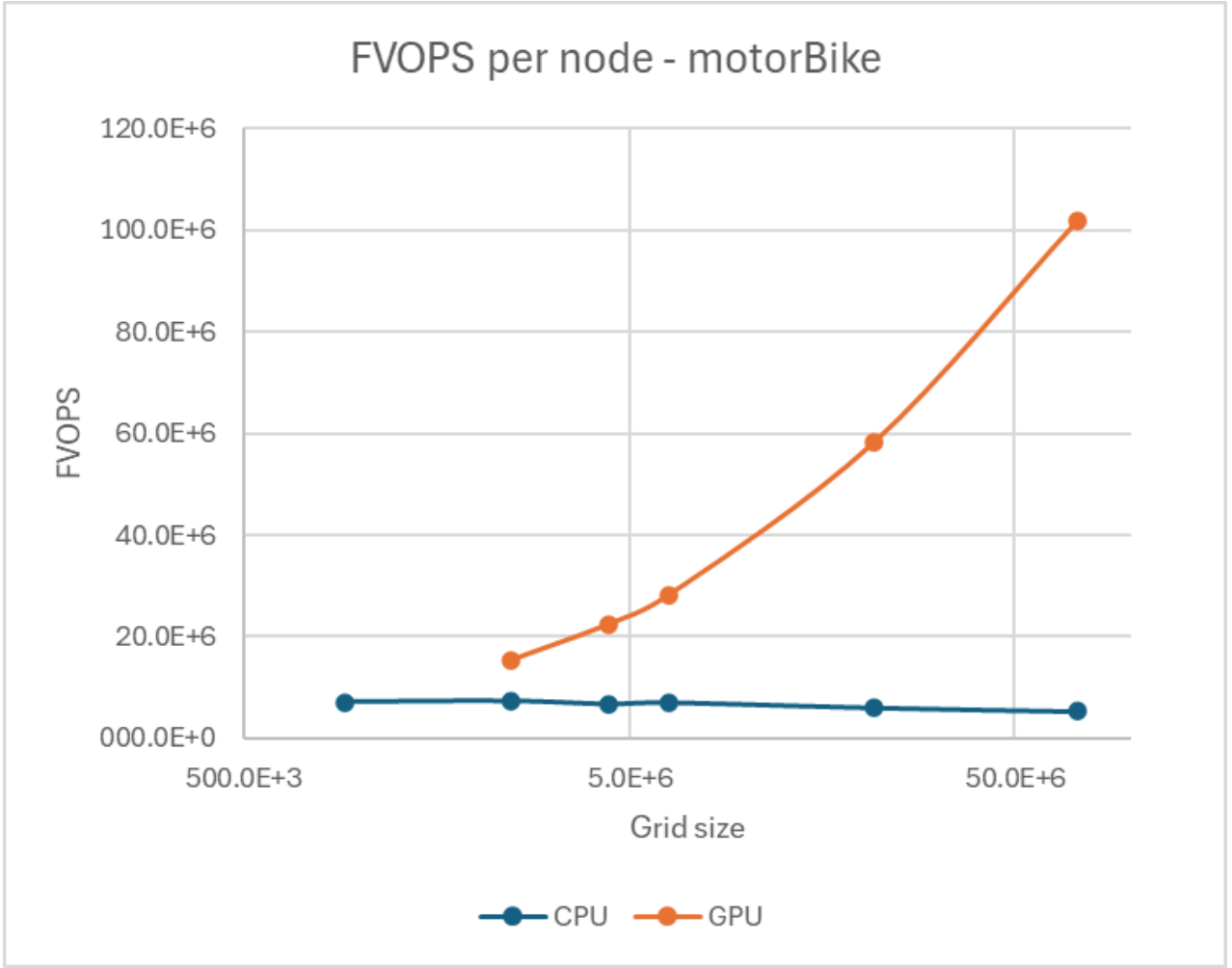
Software track – Strong scaling

- The strong scaling test is limited to the 236M case with one and two nodes
- Efficiency is ok, but not great, at 82%
- Why is the strong scaling so difficult with GPUs?



Software track – Strong scaling

- I used then the motorBike test case, as an approximation.
- With CPU, the performance does not vary much with the grid size I have tested
- With GPU, the performance increases significantly with the grid size: implications for strong scaling



Software track

- Key performance indicators
- For the 236M grid
- 10.4x faster time to solution
- 2.23x less energy to solution (energy measured by the monitoring system)
- 2.15x less cost to solution

- Cost per node-h from Hunter: CPU 1.68 €, APU 8.15 €

Thank you

Flavio C. C. Galeazzo

flavio.galeazzo@hlrs.de